Chapter 2 — HTML Structure and Text

HTML is the skeleton of every page. This chapter covers the grammar (elements, attributes, nesting), the fixed document structure, the `<head>` **metadata** (data *about* the page — title, language, character set) that browsers, search engines and social networks read, the elements for text (headings, paragraphs, inline meaning, lists, links), and finally character references, comments and validation.

In this chapter: 2.1 Elements, tags and attributes · 2.2 Document structure and the doctype · 2.3 The `<head>` · 2.4 Headings · 2.5 Paragraphs and text blocks · 2.6 Text-level semantics · 2.7 Lists · 2.8 Links · 2.9 Character references and comments · 2.10 Validation.

Examples are in `code/`. Each `.html` file is shown as a headless-Chrome screenshot; the `.mjs` scripts parse the same files with jsdom (a Node.js library that builds a browser-like DOM without a browser): `node code/01_head.mjs`.

2.1 Elements, tags and attributes

What it is

HTML (HyperText Markup Language) **marks up** plain text: you wrap content in **tags** that say what each piece *is*.

```
<a href="https://example.com" title="Opens the example site">Visit Example</a>
```

Piece	In the example
Start tag	`<a href="https://example.com" title="Opens the example site">`
Element name	`a`
Attributes (name = value)	`href` = `https://example.com`, `title` = `Opens the example site`
Content	`Visit Example`
End tag	`</a>`

- An **element** = start tag + content + end tag (`<p>Hello</p>`) — and the DOM node it becomes; a *tag* is just the `<p>` / `</p>` text. Elements nest into a tree (the DOM, Chapter 1).
- An **attribute** adds information: `name="value"`. Quote values (double quotes by convention); names are case-insensitive but write them lower-case.
- **Void elements** have no content and no end tag: `area`, `base`, `br`, `col`, `embed`, `hr`, `img`, `input`, `link`, `meta`, `source`, `track`, `wbr`. A trailing slash (`<br />`) is allowed and ignored — an XHTML habit (the stricter XML-based HTML of the 2000s).
- **Boolean attributes** are true when present, false when absent: `<input disabled>`, `<ol reversed>`. `disabled="false"` and `required="no"` still mean *on*.
- **Nesting** must be proper: `<b><i>…</i></b>`, not `<b><i>…</b></i>`.

Global attributes (usable on any element)

Attribute	Purpose
`id`	Unique identifier: link target (`#id`), label/ARIA references (ARIA = extra accessibility attributes, Chapter 3), JS hook
`class`	Space-separated names for CSS and JS; shared by many elements
`style`	Inline CSS on one element (avoid except quick tests or dynamic values)
`title`	Advisory hover tooltip; not reliably shown on touch or read by screen readers — never the only place for essential info
`lang`	Content language (`en` , `fr-CA`) — affects pronunciation, hyphenation, fonts, translation
`dir`	Text direction: `ltr` , `rtl` (e.g. Arabic), `auto`
`hidden`	Not rendered (like `display: none`)
`tabindex`	Focus order — *focus* = the element receiving keyboard input (Chapter 3)
`data-*`	Your own data for scripts: `data-product-id="42"` → `el.dataset.productId`
`inert`	Makes a subtree unfocusable and unclickable (e.g. content behind a modal dialog)
`translate` , `spellcheck` , `contenteditable`	Translation hints, spell-check, in-place editing

Block-level and inline — how elements flow

Some elements start on a new line at full width (**block**: `p` , `h1` , `ul` , `div` , `section`); others sit inside a line (**inline**: `a` , `strong` , `em` , `span` , `code`). That is only the *default CSS* `display` *value* (Chapter 7). The spec instead uses **content categories** — *flow content* (almost everything in `body`) and *phrasing content* (text and inline markup) — and each element's **content model** says what it may contain. A `<p>` takes only phrasing content, so a `<div>` or `<ul>` inside it is invalid (the parser closes the `<p>` first). An `<a>` is *transparent* (allowed content = whatever its parent allows): it may wrap blocks (a whole card) as long as it contains no other interactive element (link, button, input).

`<div>` (block) and `<span>` (inline) are generic containers with no meaning — hooks for CSS and JS when nothing meaningful fits.

Common mistakes

- Unquoted values with spaces: `class=big red` gives class `big` and a stray attribute `red` .
- Repeating an `id` — use `class` for groups.
- `<br></br>` or `<img></img>` — void elements have no end tag.
- Choosing elements for their default look ("I need bold, so `<h3>` "). Choose by meaning; CSS handles looks.

2.2 Document structure and the doctype

What it is

Every document has the same outer shape:

```html
<!DOCTYPE html>
<html lang="en">
  <head>  <!-- information ABOUT the page: not displayed -->
  </head>
  <body>  <!-- the page itself: everything the user sees -->
  </body>
</html>
```

Part	Role
`<!DOCTYPE html>`	Must be first. Switches on **standards mode**; without it, **quirks mode** emulates old browser bugs (Chapter 1 showed a table ending up *inside* a paragraph). Case-insensitive; not an element.
`<html lang="en">`	The **root element**. `lang` sets screen-reader pronunciation, enables translation and correct hyphenation; required by WCAG (Web Content Accessibility Guidelines) success criterion 3.1.1 Language of Page.
`<head>`	Metadata: character set (how bytes map to letters), title, viewport, CSS, scripts, icons, social previews (the card shown when a link is shared).
`<body>`	Content.

The `html`, `head` and `body` tags are technically optional (the parser creates them), but always write them.

The full skeleton with a realistic head is `code/skeleton.html` :

```html
<!DOCTYPE html>
<!-- The doctype above switches the browser to standards mode. This is an HTML comment. -->
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Trail Running Basics — Mountain Club</title>
  <meta name="description" content="A beginner's guide to trail running: shoes, pacing, safety and the five trails our club runs every week.">
  <link rel="icon" href="data:image/svg+xml,<svg ...><circle ... fill='%23f97316'/></svg>">
  <link rel="canonical" href="https://club.example.org/guides/trail-running">
  <meta name="theme-color" content="#f97316">
  <meta property="og:title" content="Trail Running Basics">
  <meta property="og:image" content="https://club.example.org/img/trail-card.jpg">
  <style> body { font-family: system-ui, sans-serif; max-width: 36rem; margin: 1.5rem auto; padding: 0 1rem; line-height: 1.5; } </style>
</head>
<body>
  <h1>Trail Running Basics</h1>
  <p>Everything in <code>&lt;head&gt;</code> is information <em>about</em> the page. Everything in <code>&lt;body&gt;</code> is the page.</p>
  <p>The title, description and icon you cannot see here show up in the browser tab, in search results and in link previews.</p>
</body>
</html>
```

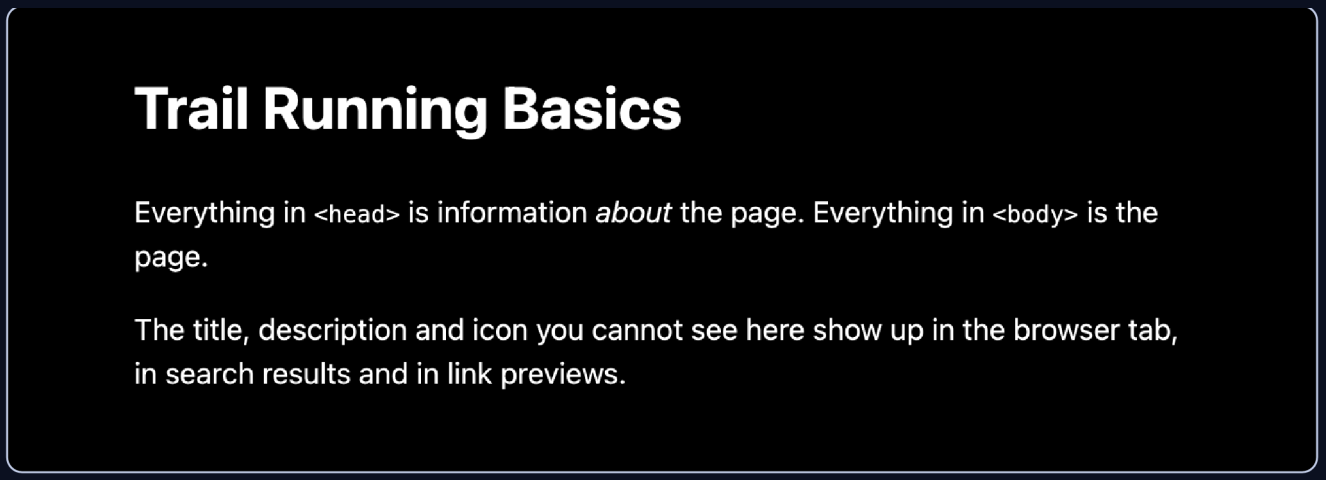


Figure 2.1 — Only the `<body>` is drawn. The head shows up elsewhere: tab title and icon, search snippets, link previews.

Old doctypes such as `<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" ...>` (DTD = Document Type Definition, the old formal grammar) mostly still trigger standards or "almost standards" mode — a few, such as HTML 4.01 Transitional without its URL, trigger quirks — so never write them today.

Self-check

1. Where does the `<title>` appear? → Tab/window title, bookmarks, history, search-result headline — not in the page body.

2.3 The `<head>`: metadata that matters

How it works

The browser reads the head top to bottom before rendering. Order matters twice: `charset` first, and render-blocking CSS (Chapter 1) early so it downloads early.

`code/01_head.mjs` parses `skeleton.html` with jsdom (as if served from `https://club.example.org/guides/trail-running`) and prints what a browser, search engine or social network learns — doctype mode, `lang`, `characterSet`, title, every `meta[name]` / `meta[property]` and `link`, comments and head children:

```
doctype   : <!DOCTYPE html> -> CSS1Compat
lang      : en
charset   : UTF-8 (declared at byte 140 — must be within the first 1024)
title     : Trail Running Basics — Mountain Club
meta      : viewport     = width=device-width, initial-scale=1
meta      : description  = A beginner's guide to trail running: shoes, pacing, safety a
meta      : theme-color  = #f97316
meta      : og:title     = Trail Running Basics
meta      : og:image     = https://club.example.org/img/trail-card.jpg
link      : icon         -> data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' v
link      : canonical    -> https://club.example.org/guides/trail-running
comments  : [
  'The doctype above switches the browser to standards mode. This is an HTML comment.'
]
head children: meta, meta, title, meta, link, link, meta, meta, meta, style
```

The essential head elements

Element	Example	Why
`<meta charset="utf-8">`	—	How to turn bytes into characters. Must be within the **first 1024 bytes**. Use UTF-8 (the universal encoding covering every language and emoji) only; otherwise accents turn to garbage ("cafÃ©"). An HTTP `Content-Type: …; charset=` header overrides it.
`<meta name="viewport">`	`content="width=device-width, initial-scale=1"`	Without it phones lay out at ~980 px and shrink the page. Makes CSS pixels (logical pixels, possibly several device dots each) match the device width — the foundation of responsive design (Chapter 10). Never add `maximum-scale=1` or `user-scalable=no`: blocking zoom fails WCAG 1.4.4 (Resize Text).
`<title>`	`Trail Running Basics — Mountain Club`	Required. Unique per page, most specific part first. First thing a screen reader announces; the search-result headline.
`<meta name="description">`	one or two sentences	Often the search snippet (engines may pick other text), usually cut at ~150–160 characters. Not a ranking factor — a pitch for the click. Not visible on the page.
`<link rel="stylesheet">`	`href="css/style.css"`	Loads CSS (render-blocking).
`<script src defer>` / `type="module"`	`src="js/app.js"`	Loads JS without blocking the parser (Chapter 1).
`<link rel="icon">`	`.ico`, `.png` or `.svg` (Chapter 4)	The tab icon (favicon). Add `rel="apple-touch-icon"` for iOS home screens.
`<link rel="canonical">`	full URL	The preferred ("official") URL when the same content lives at several addresses.
Open Graph (Facebook-originated) / Twitter Card metas	`property="og:title"`, `og:description`, `og:image`	The preview card on social networks and chat apps.
`<meta name="theme-color">`	`#f97316`	Tints the browser UI on some mobile browsers.
`<meta name="robots">`	`content="noindex"`	Keeps a page out of search results.
`<base href>`	`href="/docs/"`	Changes the base for *all* relative URLs. Rarely a good idea — it surprises people.
`<link rel="preload">`, `rel="preconnect"`	fonts, hero image, CDN origin	"Fetch this early" / "connect early" hints (Chapter 14).

Keep a copy-paste **template**: doctype, `lang`, charset, viewport, title, description, stylesheet, deferred script.

Common mistakes

- `<meta charset>` after a long comment or a non-ASCII `<title>` (é, emoji…) — it may fall outside the first 1024 bytes.
- The same `<title>` ("Home") on every page.
- Visible content in the head: the parser moves it into the body.
- Disabling zoom in the viewport tag.

2.4 Headings

What it is

`<h1>` … `<h6>` are section headings, `h1` most important; together they form the page's **outline**.

Why it matters

- **Screen-reader users** navigate by headings: in WebAIM's (a US accessibility non-profit) surveys, a large majority say jumping through headings is how they first find information on a long page (Chapter 3).
- **Everyone** skims headings; **search engines** use them to understand each part of the page.



Figure 2.2 — `code/text-blocks.html`: the six heading levels at default sizes, then the block text elements from §2.5.

Rules that work

1. **One `<h1>` per page**, describing it (usually close to the `<title>`). The spec allows more; one is clear best practice.
2. **Don't skip levels going down** (`h2` → `h3`, not `h4`). Going back up any number (`h4` → `h2`) is fine — it starts a new major section.
3. **Choose the level by structure, never size.** Want a small `h2`? Use CSS.
4. **Don't use headings for non-headings** (a big slogan is a `<p>` with a class).
5. Heading + tagline: `<hgroup><h1>Title</h1><p>Tagline</p></hgroup>` — so the tagline isn't a fake heading.

There is no "document outline algorithm". Old HTML5 drafts let each `<section>` restart heading levels (so every section could use `h1`). No browser or screen reader implemented it and the WHATWG removed it in 2022: heading levels are what the numbers say, and `<section>` doesn't change them.

Common mistakes

- `<h4>` because it "looks right" — a screen reader announces "heading level 4" and users assume missing sections.
- Several `h1`s in a template (logo *and* page title).
- `<div class="title">` headings — they appear in no heading list.

2.5 Paragraphs, whitespace and text blocks

What it is

Element	Meaning	Notes
`<p>`	A paragraph	Don't use empty `<p>`s for spacing — use CSS `margin`.
` `	A line break that is part of the content	Addresses, poems, lyrics. Never for layout spacing.
`<hr>`	A thematic break (topic or scene change)	Drawn as a line; may be announced as a separator. Decorative lines → CSS border.
`<pre>`	Preformatted text: whitespace and line breaks kept, monospace font	Code blocks (`<pre><code>…</code></pre>`), ASCII art
`<blockquote>`	A long quotation from another source	Optional `cite="URL"` (not shown); visible attribution outside it or in a `<figcaption>` (Chapter 4)
`<address>`	Contact information for the nearest `<article>` or the page	Not for any postal address in text
`<div>`	Generic block container, no meaning	When nothing else fits

Whitespace collapsing

In normal text any run of spaces, tabs and newlines **collapses** to one space, and leading/trailing whitespace in a line is dropped — so you can indent HTML freely. To keep whitespace use `<pre>` or CSS `white-space`; for a space that never breaks a line use ` ` (§2.9).

2.6 Text-level semantics

What it is

Inline elements that say what a *phrase* is. Most have a default look, but choose by meaning, then style with CSS.

Element	Rendered example
strong	Do **not** unplug during the update.
em	I *said* tomorrow, not today.
b	Ingredients: **flour**, **eggs**, **milk**.
i	The ship *Titanic*; the term *déjà vu*.
mark	Results for "grid": CSS grid layout
small	© 2026 Example Ltd. Terms apply.
s	Price: ~~$40~~ $25
del / ins	Meet on ~~Monday~~ Tuesday.
sub / sup	H_2O, $E = mc^2$, 1st
code	Call `fetch()` to request data.
kbd	Press `Ctrl` + `C` to copy.
samp	The terminal printed `Permission denied`.
var	The area is *w* × *h*.
abbr	HTML (hover for the expansion)
dfn	A *viewport* is the visible area of the page.
cite	As described in *The Pragmatic Programmer*.
q	She said "ship it" and left.
time	Doors open Saturday at 7 pm.
data	Product Blue mug
u	A speling mistake (unarticulated annotation)
bdi	User ياسر: 3 posts
wbr	supercalifragilisticexpialidocious
span	A span has no meaning — a styling hook only.

Figure 2.3 — `code/inline-semantics.html` : every common text-level element with its default rendering.

Element	Means	Use for	Not for
`<strong>`	Strong importance, seriousness or urgency	Warnings, key facts	Headings, general "make it bold"
`<em>`	Stress emphasis that changes the meaning	"I *said* tomorrow"	Titles, foreign words
`<b>`	Draw attention without extra importance	Keywords, product names in a review, lead sentence	Importance (use `strong`)
`<i>`	Different voice or mood	Technical terms, foreign phrases (`<i lang="fr">`), ship names, thoughts	Emphasis (use `em`), icons
`<mark>`	Highlighted for relevance to the current context	Search matches	Style-only highlighting
`<small>`	Side comments, fine print	Copyright, legal notes	Making text small for design
`<s>`	No longer accurate or relevant	Old price	Document edits (use `del`)
`<del>` / `<ins>`	Removed / added text in an edit	Change tracking; optional `datetime`, `cite`	—
`<sub>` / `<sup>`	Subscript / superscript required by typography	H_2O, footnote markers, 1st	Pure visual offsets
`<code>`	Computer code fragment	Inline code; blocks in `<pre><code>`	—
`<kbd>`	User input (keys, commands)	Ctrl+C	—
`<samp>`	Sample program output	Error messages	—
`<var>`	A maths/programming variable	x, $w \times h$	—
`<abbr>`	Abbreviation; `title` gives the expansion	First use of "HTML" — also write the expansion in text, since `title` is not reliably exposed	—
`<dfn>`	The defining instance of a term	Where a term is first defined	Later mentions
`<cite>`	Title of a creative work	Books, films, papers	People's names
`<q>`	Short inline quotation; browser adds quote marks	"She said *ship it*"	Long quotes (`blockquote`)
`<time>`	Date/time with a machine-readable `datetime`	`<time datetime="2026-10-03T19:00">Saturday at 7 pm</time>`	—
`<data>`	Content with a machine-readable `value`	Product IDs, numbers	Dates (use `time`)
`<u>`	Unarticulated, non-textual annotation	Spelling-error squiggles, Chinese proper-name marks	Normal text (looks like a link)
`<bdi>`	Isolates text of unknown direction	User names that may be Arabic/Hebrew in an LTR page	—
`<wbr>`	Where a long word *may* break	Long URLs, compound words	—
`<span>`	Nothing — styling/scripting hook	When nothing above fits	—

Why it matters

`<time datetime>` lets calendars and search engines read dates; `<code>` stays monospace in reader modes; `<abbr>`, `<dfn>` and `lang` help translation and screen readers. Most screen readers **do not** announce `<strong>` / `<em>` by default, so state critical information in words too ("Warning: …").

Common mistakes

- `<i class="icon-star">` for icons — a legacy icon-font habit; use `<span aria-hidden="true">` (hidden from screen readers) or an SVG.
- Whole paragraphs in `<strong>` : if everything is important, nothing is.
- `<cite>` around a person's name.

2.7 Lists

What it is

Element	Use when	Items
`<ul>`	Order doesn't matter (shopping list, nav links, features)	`<li>`
`<ol>`	Order matters (steps, rankings, legal clauses)	`<li>`
`<dl>`	Name–value groups (glossary, metadata, FAQ)	`<dt>` term(s) + `<dd>` description(s); groups may be wrapped in a `<div>`
`<menu>`	A toolbar of commands	Semantically the same as `<ul>`

`<ol>` attribute	Effect
`start="4"`	First number is 4
`reversed`	Counts down
`type="1	a
`<li value="10">`	Sets this item's number; later items continue from it

```
<!-- code/lists.html (excerpt) -->
<ul>
  <li>Milk</li>
  <li>Bread
    <ul><li>Sourdough</li><li>Rye</li></ul>   <!-- nested list goes INSIDE the <li> -->
  </li>
  <li>Apples</li>
</ul>
<ol start="4" type="a"> <li>fourth item is "d"</li> <li>then "e"</li> </ol>
<ol reversed> <li>Bronze</li> <li>Silver</li> <li>Gold</li> </ol>
<ol type="I"> <li>Introduction</li> <li value="10">Jumps to X</li> <li>Then XI</li> </ol>
<dl>
  <dt>HTML</dt><dd>Structure and meaning</dd>
  <dt>CSS</dt><dd>Presentation</dd>
  <dt>JS</dt><dt>JavaScript</dt><dd>Behaviour (two terms, one description)</dd>
</dl>
```

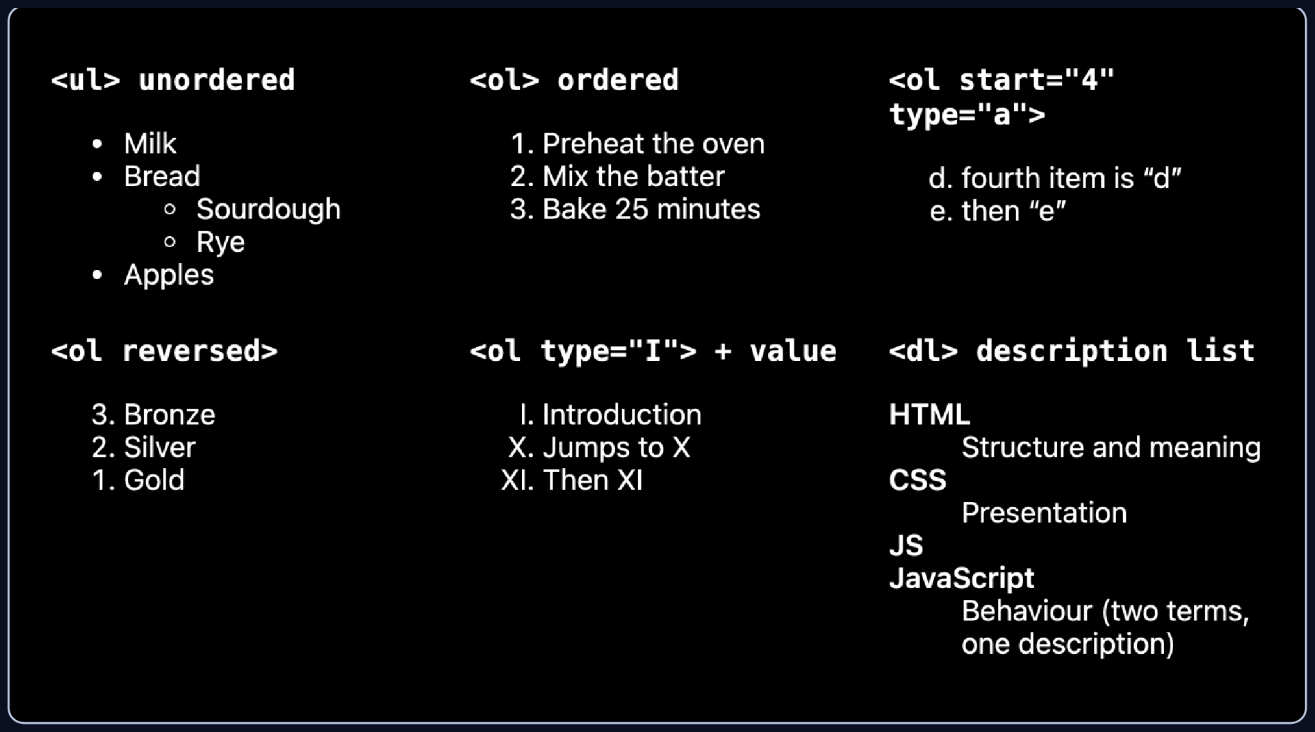


Figure 2.4 — `code/lists.html` in Chrome.

Why it matters

Screen readers announce "list, 3 items" and let users jump between lists and items. Navigation menus are usually `<nav><ul>…</ul></nav>`.

Common mistakes

- Anything other than `<li>` directly inside `<ul>` / `<ol>` (a `<p>`, `<div>`) — invalid; list semantics break.
- A sub-list as a sibling of `<li>` instead of inside one.
- Fake lists with `<br>` and "•".
- `list-style: none` — then Safari/VoiceOver (Apple's screen reader) stops announcing a list; add `role="list"` (ARIA, Chapter 3) if the count matters.

2.8 Links

What it is

`<a href="…">` — the *anchor* — is the "hyper" in hypertext. Its content (text, image, a whole card) becomes clickable and keyboard-focusable; activating it navigates to the `href`.

```
<!-- code/links.html (excerpt) -->
<a href="https://developer.mozilla.org/en-US/docs/Web/HTML">MDN HTML docs</a>        <!-- absolute -->
<a href="text-blocks.html">Headings and text</a>                                      <!-- document-relative -->
<a href="../index.html">All guides</a>                                               <!-- up one folder -->
<a href="#faq">Jump to the FAQ</a>                                                   <!-- fragment: element with
id="faq" -->
<a href="mailto:help@example.org?subject=Question">Email us</a>
<a href="tel:+15555550123">Call +1 555 555 0123</a>
<a href="skeleton.html" download="page-template.html">Download the template</a>
<a href="https://example.com/" target="_blank" rel="noopener noreferrer">Example.com (opens in new tab)</a>
<a href="https://example.com/sponsor" rel="sponsored">A paid link</a>
<a>Placeholder link (no href)</a>
...
<h2 id="faq">FAQ</h2>
```

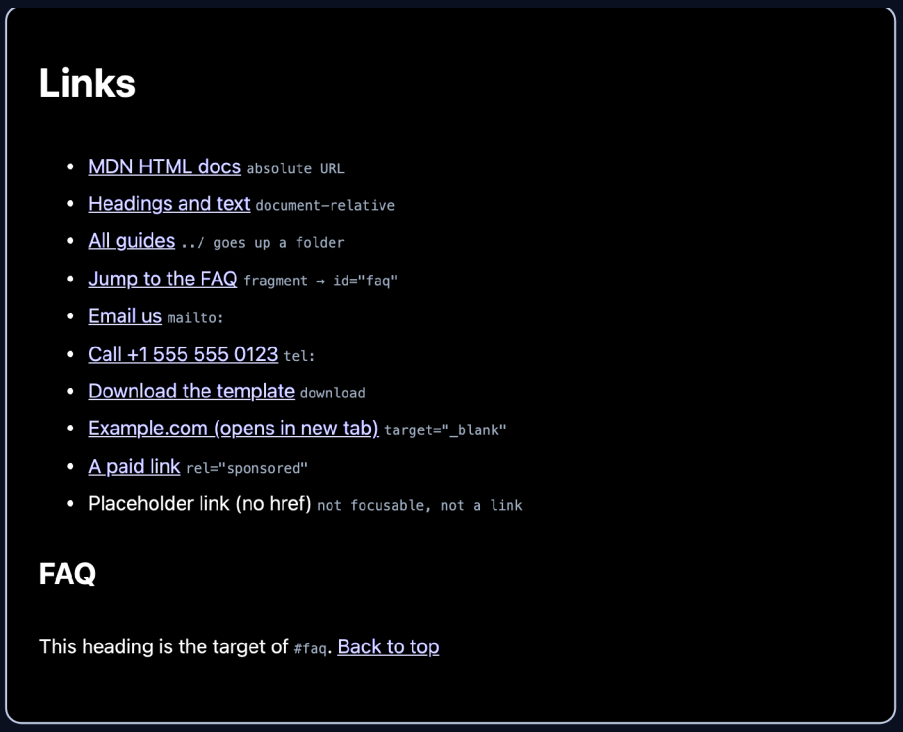


Figure 2.5 — `code/links.html`. The `<a>` without `href` is not styled or focusable as a link.

`code/02_links.mjs` loads the file as if it lived at `https://club.example.org/guides/html/links.html` and prints how each `href` resolves:

```
attribute vs property: text-blocks.html -> https://club.example.org/guides/html/text-blocks.html

MDN HTML docs              https://developer.mozilla.org/en-US/docs/Web/HTML       https:
Headings and text          https://club.example.org/guides/html/text-blocks.html   https:
All guides                 https://club.example.org/guides/index.html              https:
Jump to the FAQ            https://club.example.org/guides/html/links.html#faq     https: same-page jump to #faq
Email us                   mailto:help@example.org?subject=Question                mailto:
Call +1 555 555 0123       tel:+15555550123                                        tel:
Download the template      https://club.example.org/guides/html/skeleton.html      https: download=page-
template.html
Example.com (opens in new  https://example.com/                                    https: target=_blank
rel=noopener noreferrer
A paid link                https://example.com/sponsor                             https: rel=sponsored
Placeholder link (no href) (no href: not in document.links = false)
Back to top                https://club.example.org/guides/html/links.html#top     https: same-page jump to #top
```

The `href` *attribute* holds what you typed; the `.href` *property* is the resolved URL (§1.2 rules). `href="#faq"` scrolls to `id="faq"` with no request.

Link attributes

Attribute	What it does	Notes
`href`	Destination URL	Without it `<a>` is a placeholder: not a link, not focusable
`target="_blank"`	Open in a new tab/window	Others: `_self` (default), `_parent`, `_top`, a named window
`rel`	Relationship of the target to this page	Space-separated, see below
`download`	Download instead of navigate; value = suggested file name	Only same-origin, `blob:` (in-memory file made by JS) or `data:` URLs; ignored cross-origin (navigates normally)
`hreflang`, `type`	Hints about the target's language / MIME type (format label such as `text/html`)	Advisory
`referrerpolicy`	How much of this URL to send in `Referer`	e.g. `no-referrer`
`ping`	URLs to notify when followed	Tracking; not in every browser

`rel` value	Meaning
`noopener`	The new page gets no `window.opener` (a JS handle back to your page), so it cannot redirect your tab to a phishing page ("reverse tabnabbing"). Modern browsers apply it automatically to `target="_blank"`; writing it is harmless.
`noreferrer`	Also don't send `Referer` (implies `noopener`)
`nofollow`	"I don't vouch for this link" — a hint to search engines
`sponsored`	Paid or affiliate link (search engines ask you to mark these)
`ugc`	User-generated content link (comments, forums)
`external`, `author`, `license`, `next`, `prev`	Descriptive relations

Opening new tabs — should you?

Mostly no: forcing one breaks Back and can disorient screen-reader and mobile users, who can open tabs themselves. Exceptions: links that would interrupt a form or playing video, help pages beside a task. Then say so ("(opens in new tab)" or an icon with a text alternative).

Links vs buttons

A **link** goes somewhere (a URL; Enter activates). A **button** does something on this page (submit, toggle, open a menu; Enter or Space). `<a href="#" onclick="…">` fakes a button: it adds `#` to the URL, jumps to the top without JS, and is announced as "link". Use `<button type="button">` (Chapter 3).

Writing good link text

Link text must make sense out of context — screen-reader users often list all links, where "Click here", "read more" and bare URLs mean nothing. Write "Download the 2026 price list (PDF, 2 MB)". This is WCAG 2.4.4 (Link Purpose (In Context)).

Real-world examples

- "Skip to main content" links are fragment links (`href="#main"`) — Chapter 3.
- GitHub and MDN give every heading an `id` for deep links; shops use `tel:` for one-tap calls.

Common mistakes

- Icon-only links with no text — no accessible name (the label a screen reader announces).
- `href="www.example.com"` without `https://` — a *relative* path (`/current/www.example.com`).
- Two adjacent links to the same place (card image + title) — noisy for keyboard and screen-reader users; wrap both in one link.

In the interview

> **30-second answer:** "An anchor with an `href` navigates to a URL — absolute, relative, fragment or another scheme like `mailto:` . I keep link text meaningful out of context, never use `href="#"` for actions because those are buttons, and use `target="_blank"` sparingly. `rel` describes the relationship: `noopener` / `noreferrer` for security and privacy, `nofollow` / `sponsored` / `ugc` for search engines."

- *Why was `rel="noopener"` needed with `target="_blank"`?* → The opened page could set `window.opener.location` to swap your tab for a phishing page. Browsers now imply `noopener` for `_blank`.
- *Link or button for "Add to cart"?* → Button: an action on the current page.
- *`download` on a cross-origin URL?* → Ignored; the browser navigates normally.

2.9 Character references and comments

What it is

`<` starts a tag and `&` starts a **character reference**, so to show them as text you write a reference:

- **Named**: `<` `>` `&` `"` ` ` `©` (often called "entities"; strictly *named character references*)
- **Decimal**: `<` — the Unicode *code point* (each character's number in the Unicode catalogue)
- **Hexadecimal**: `<`

Shows	Named	Numeric	Why you need it
<	<	<	Would start a tag
>	>	>	Pairs with < for readability
&	&	&	Would start a character reference
"	"	"	Inside a "double-quoted" attribute
'	'	'	Inside a 'single-quoted' attribute
a b			Space that never breaks: 10 km
©	©	©	Hard to type (or just type ©)
—	—	—	Em dash
€	€	€	Hex form uses x
			Invisible soft hyphen (break hint)

Figure 2.6 — `code/entities.html` : what each reference renders as.

With `<meta charset="utf-8">` you can type almost any character directly (©, —, €, é, emoji). You only *need* references for:

Character	Where	Reference
`<`	In text	`<` (so the text ` ` is ` `, or set via `textContent`)
`&`	In text or attributes, when followed by something that looks like a reference	`&` (always escaping is simplest)
`"` / `'`	Inside an attribute quoted with the same character	`"` / `'`
Invisible characters	Non-breaking space, soft hyphen, zero-width characters	` `, ``, `` — visible in your source

```
// code/03_entities.mjs
import '../../../../tools_dom.mjs';
const p = document.createElement('p');
p.innerHTML = 'Fish &amp; chips &lt;b&gt; &copy; &#169; &#xA9; 10 km &mdash; café';
console.log('textContent :', JSON.stringify(p.textContent));
console.log('nbsp code   :', p.textContent.codePointAt(p.textContent.indexOf('km') - 1).toString(16));
console.log('innerHTML   :', p.innerHTML);
const q = document.createElement('p');
q.textContent = '<img src=x onerror=alert(1)> & "quotes"';     // text, not HTML
console.log('\nescaped     :', q.innerHTML);
console.log('elements    :', q.children.length, '(the tag became text, not an element)');
const r = document.createElement('p');
r.innerHTML = 'AT&T, R&D, a&b=1';
console.log('\nbare &      :', r.textContent);
const s = document.createElement('div');
s.innerHTML = '<p>price&copy=1</p><a href="?a=1&copy=2">x</a>';
console.log('in text     :', s.querySelector('p').textContent, '| in href:',
s.querySelector('a').getAttribute('href'));
```

Output:

```
textContent : "Fish & chips <b> © © © 10 km — café"
nbsp code   : a0
innerHTML   : Fish &amp; chips &lt;b&gt; © © © 10 km — café

escaped     : &lt;img src=x onerror=alert(1)&gt; &amp; "quotes"
elements    : 0 (the tag became text, not an element)

bare &      : AT&T, R&D, a&b=1
in text     : price©=1 | in href: ?a=1&copy=2
```

1. The parser **decodes** references; `©`, `©` and `©` are the same `©`.
2. ` ` is the real character U+00A0 (`a0`), not a space — keeps "10 km" or "Mr. Smith" on one line.
3. When **serialising** (DOM → HTML text, e.g. reading `innerHTML`) the browser re-escapes `&`, `<`, `>` and non-breaking spaces.
4. Setting `textContent` is safe: markup becomes text. Assigning user input to `innerHTML` is how **cross-site scripting (XSS)** happens — an attacker's script runs in your page (JavaScript book).
5. A bare `&` usually survives but is fragile: ~100 legacy names such as `&copy` decode even *without* a semicolon, so `price&copy=1` shows `price©=1`. In attributes a special rule leaves `&copy=` alone, which is why most query strings survive. `&` removes all doubt.

Comments

```
<!-- This is a comment. It can span
     several lines. It is not displayed. -->
```

Comments stay in the DOM as comment nodes (`01_head.mjs` found one) and are **visible to anyone** via View Source — never put passwords, internal URLs or rude notes in them. They cannot nest; avoid `--` inside. Build tools usually strip them from production HTML.

2.10 Validation

The problem it solves

Browsers never show HTML errors — they silently repair them (Chapter 1), because the spec requires recovery. A broken page can *look* fine while behaving differently for screen readers, search engines, other browsers, or your CSS and JS.

A page with a dozen problems (code/broken.html):

```
<html>
<head>
<title>Broken page</title>
</head>
<body>
<h1>Welcome</h1>
<h3>Our services</h3>
<p>Great deals <b>today <i>only</b></i>
<p id="promo">Sale!</p>
<p id="promo">Another sale!</p>
<img src="data:image/svg+xml,...">
<a href="#" onclick="go()">Click here</a>
<ul><p>Not a list item</p><li>ok</li></ul>
<h1>Also welcome</h1>
</body>
</html>
```

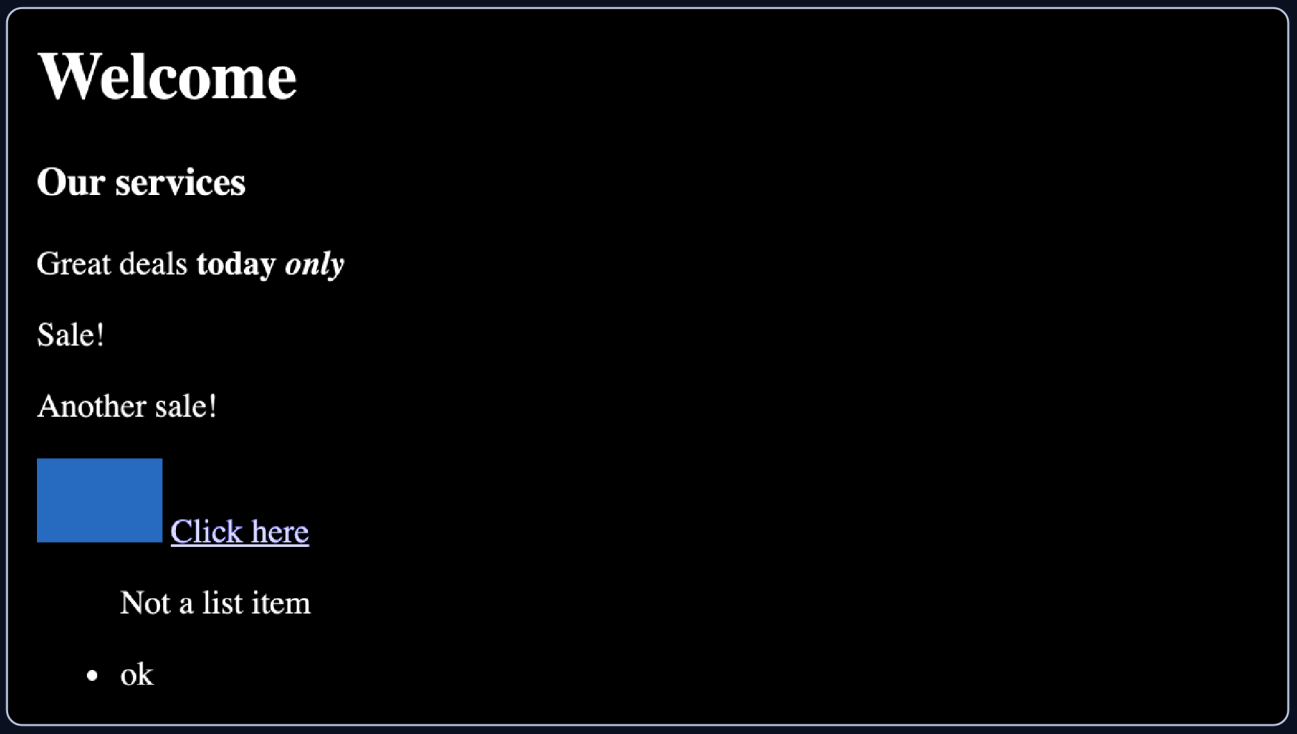


Figure 2.7 — It renders without complaint. Nothing on screen reveals the 12 problems below.

How it works — checking

1. **The W3C Nu HTML Checker** (validator.w3.org/nu) — the reference validator (paste, upload or URL). Reports **parse errors** (bad nesting, stray end tags) and **content-model errors** (`<p>` in `<ul>` , duplicate ids, missing required attributes such as `alt`).
2. **Editor and build tools**: VS Code extensions, `html-validate` or `vnu` (the Nu checker's CLI) in CI (continuous integration — automated checks on every push).
3. **Accessibility checkers** (axe, Lighthouse, WAVE; Chapter 3) catch what validators don't: empty links, poor contrast.

code/04_validate.mjs is a tiny jsdom checker for a dozen common problems (doctype, `lang` , charset within 1024 bytes, viewport, title, `h1` count, heading jumps, duplicate ids, `img` without `alt` ,

`href="#"` / `javascript:` links, vague link text, non-`li` list children, `<b><i></b>` mis-nesting):

```
skeleton.html: OK
broken.html: 12 problem(s)
  - missing <!DOCTYPE html> -> quirks mode
  - <html> has no lang attribute
  - no <meta charset> in the first 1024 bytes
  - no viewport meta (page will not scale on phones)
  - 2 <h1> elements (aim for exactly one)
  - heading jumps from h1 to h3: "Our services"
  - duplicate id="promo"
  - <img> without alt
  - fake link "Click here" (use <button>)
  - vague link text "Click here"
  - <p> directly inside <ul>
  - mis-nested tags <b><i>...</b> (parser repaired it)
```

Some are validity errors (duplicate id, `<p>` in `<ul>`, missing `alt`, missing doctype/title); others are best-practice or accessibility warnings (heading jumps, two `h1`s, vague link text). A validator plus an accessibility checker covers both.

Trade-offs

Validity isn't the goal — a valid page can still be inaccessible. But valid HTML is predictable across browsers and tools, and most errors are real bugs. Treat errors as bugs, warnings as review items.

In the interview

30-second answer: "The HTML parser repairs almost any markup, so errors are invisible. That's why I validate — the repaired DOM may not be what I meant, and assistive technology (screen readers, switch devices), crawlers and my CSS selectors all see the repaired tree. I use the Nu HTML Checker or html-validate in CI, plus axe or Lighthouse for accessibility."

- *Name three common validation errors.* → Duplicate ids, block elements inside `<p>`, missing `alt` on `<img>` (also bad nesting, non-`li` children in lists).
- *Does invalid HTML affect SEO (search engine optimisation)?* → Not directly as a ranking factor, but broken structure can hide content, headings or links from crawlers.
- *Does a valid page guarantee accessibility?* → No — add accessibility testing.

Chapter 2 TL;DR

- **Elements** = tags + content; attributes add information; void elements have no end tag; boolean attributes are on when present. Nest properly.
- Every page: `<!DOCTYPE html>` → `<html lang>` → `<head>` (charset first, viewport, unique `<title>`, description, CSS, deferred JS, icon, canonical, social tags) → `<body>`.
- **Headings** build the outline: one `h1`, don't skip levels, choose by structure not size. There is no outline algorithm for `<section>`.
- **Text blocks**: `p` for paragraphs, `br` only for meaningful breaks, `pre` keeps whitespace, `blockquote` for long quotes; whitespace otherwise collapses.
- **Inline semantics**: `strong` (importance) vs `b` (attention), `em` (stress) vs `i` (other voice), plus `mark`, `code`, `kbd`, `time`, `abbr`, `cite`, `q` … `span` / `div` have no meaning.
- **Lists**: `ul` unordered, `ol` ordered (`start`, `reversed`, `type`, `value`), `dl` for name–value pairs; nested lists go inside `li`.
- **Links**: resolve like URLs; meaningful text; `href="#"` actions should be buttons; `target="_blank"` sparingly (implies `noopener`); `rel` for security, privacy and SEO hints.
- **References**: escape `<` and `&` (`<`, `&`), ` ` for non-breaking space; `textContent` is safe, raw `innerHTML` is an XSS risk. Comments are public.

- **Validate**: browsers silently repair errors; use the Nu checker/html-validate plus accessibility tools.

Next: Chapter 3 — Semantic HTML and Accessibility.